Nishant Trivedi

Software Engineer — Backend & Full Stack

linkedin.com/in/nishant474 github.com/nishant474 nishant-trivedi.netlify.app nishanttrivedi474@gmail.com
Surat, Gujarat

Profile

Software Engineer who builds **scalable backend systems** and **fast, reliable frontends**. Currently modernizing a **legacy C# financial data platform into event-driven Python / FastAPI microservices** on Azure. Strong in **Python, Node.js, React, PostgreSQL** and distributed messaging.

Skills

LANGUAGES
Python, JavaScript, Java, SQL, C# (legacy)

BACKEND
FastAPI, asyncio, SQLAlchemy, Alembic, Node.js, Express.js, REST APIs, JWT

DATA & MESSAGING
PostgreSQL, SQL Server, Redis, RabbitMQ, Azure Service Bus, BullMQ, Polars, Parquet

FRONTEND
React, Redux Toolkit, React Query, Formik, Tailwind CSS, HTML, CSS

CLOUD & DEVOPS
Docker, Azure (App Service, Functions, File Share), Playwright, Selenium

Achievements

Employee of the Month, *Engross Infotech*
04/2025
Collaborated with **cross-functional teams** to ship **scalable features on time**.

Ignite — Winner | Charusat University
01/2023
1st place in the university-level coding competition **Code Nigma**, CMPICA, Charusat.

Academic Performer | VNSGU
01/2022
Top performer in BCA at VNSGU; awarded a **merit scholarship**.

Education

Master of Computer Applications
Charusat University — CGPA 8.14
08/2022 – 04/2024 | Nadiad, India

Bachelor of Computer Applications
VNSGU — CGPA 9.17
07/2019 – 03/2022 | Surat, India

Professional Experience

Software Engineer — Full Stack, *Engross Infotech*
06/2024 – present | Surat, India

Financial Data Ingestion Platform (C# → Python) 06/2026 – present

- **Sole engineer:** architected and built the migration of a **240K-line legacy C#/.NET ingestion monolith** to **14 event-driven Python FastAPI microservices** (~115K LOC) on Azure in 5 months, reaching **100% ledger parity**, validated by **3 weeks of daily side-by-side runs against the legacy system** on 28 production feeds (2 clients, ~10K accounts).
- Designed a **6-stage async pipeline** (ingest → validate → transform → reconcile → FX → persist) over **RabbitMQ / Azure Service Bus**, with a dual-transport messaging library and **per-account dead-letter isolation** so one bad account never fails a batch.
- Built a **lease-based worker scheduler** on PostgreSQL (**FOR UPDATE SKIP LOCKED**, heartbeats) that auto-reclaims crashed jobs, plus stage checkpoints for **single-account replay**.
- Added **multi-tenant database isolation** via dynamic DSN routing and capped **asyncpg / SQLAlchemy** pools, eliminating connection exhaustion and cross-client lock contention.
- Replaced JSON blobs with **Parquet + Polars** staging (**80–90% smaller payloads**); screening 3,000+ accounts in **under 2s** at intake cut downstream load by **95%**.
- Removed a reconciliation bottleneck by **batching 3 queries into 1** and eliminating PostgreSQL catalog lock contention; automated **row-by-row parity audit reports**.

Harvesting Scraping Module

- Migrated an **RPA-based Azure Function** backend to a scalable **Node.js / Express REST API** with concurrency control across parallel scraping tasks, in **Dockerized isolated environments**.
- Implemented **rate-limit handling, retries and anti-bot mitigation**, and automated extraction with **Selenium, Playwright** and HTTP request mimicking with secure credential handling.

Alts — Reimagined

- Designed a **persistent background job system** with **BullMQ + Redis** supporting delayed, recurring and fault-tolerant tasks (Hangfire-like).
- Replaced generic data layers with **custom REST APIs and optimized SQL / stored procedures**, improving response times by **~50%**.
- Optimized rendering with **virtualization and memoization**; built server-state with **React Query** and scalable state with **Redux Toolkit**.
- Hardened frontend security: **XSS, CSRF** and secure cookie management.

Java Developer, *Knovos India Pvt. Ltd*
01/2024 – 04/2024 | Gandhinagar, India

- Used **Core Java multithreading** and the Collections Framework for parallel processing, **improving performance by 40%**.
- Used **Hadoop with Solr** for parallel processing of pretrained **AI/ML models** and indexed sentiment results.